ISSUE 03 · NOVEMBER 2026 SLOT

Finding the cryptography inside an OPC UA deployment

Friday Ogochukwu Ikwuogu · ORCID 0009-0009-2222-1318 · Published 2026-11-03 · 6 min read

The first issue of this series argued that a pipeline operator's post-quantum work starts with an inventory, and that certificates are the easiest place to begin. OPC UA is where that advice gets tested, because OPC UA is the OT protocol most likely to be doing real public-key cryptography in your environment already.

Many pipeline control centers use OPC UA between historians, HMIs, analytics platforms, and gateways that front older field protocols. Unlike Modbus or classic DNP3, OPC UA was designed with security built in: every application has its own certificate, every secure channel negotiates keys, and every connection names the cryptographic suite it uses. That makes it unusually inventory-friendly, if you know where to look.

How OPC UA uses public-key cryptography

An OPC UA client and server set up a *secure channel* before they exchange data. Three things in that exchange matter for quantum readiness.

1. **The security policy.** Each endpoint advertises one or more security policies, identified by a URI. The policy fixes the algorithms for the whole channel: how keys are exchanged, how messages are signed, and which symmetric cipher protects the data.

2. **The security mode.** `None`, `Sign`, or `SignAndEncrypt`. A policy only protects the channel if the mode uses it.
3. **The application instance certificates.** Each client and server presents an X.509 certificate. The OpenSecureChannel handshake uses the public keys in those certificates to establish the channel's symmetric keys and to authenticate each side.

The established policies are RSA-based:

SECURITY POLICY	PUBLIC-KEY OPERATIONS	STATUS
Basic128Rsa15	RSA with PKCS#1 v1.5 padding, SHA-1	Deprecated
Basic256	RSA-OAEP, SHA-1	Deprecated
Basic256Sha256	RSA-OAEP, RSA signatures with SHA-256	Current, most widely supported
Aes128_Sha256_RsaOaep	RSA-OAEP, SHA-256	Current
Aes256_Sha256_RsaPss	RSA-OAEP, RSA-PSS signatures, SHA-256	Current

OPC UA 1.05 added elliptic-curve policies (ECC_nistP256, ECC_nistP384, and Brainpool curve variants), which give smaller keys and faster handshakes. Support in shipping products is still uneven.

Every one of these policies relies on RSA or elliptic-curve cryptography for key establishment and authentication. All of it is quantum-vulnerable. OPC UA has no standardized post-quantum security policy yet, so the practical goal today is not to migrate OPC UA. It is to know exactly where each policy is in use, so that you can move quickly when one exists and remove the deprecated ones now.

Four places to look

1. Endpoint configuration

Every OPC UA server publishes its endpoints, and each endpoint pairs a security mode with a security policy. Any OPC UA client that can browse endpoints will list them without opening a session. Record, for each server:

- every endpoint URL;
- the security modes offered, flagging any `None` endpoint on a production server;
- the security policies offered, flagging Basic128Rsa15 and Basic256;
- whether the server accepts anonymous users.

The server offering a strong policy is not the same as clients using it. A server that also offers `None` or a deprecated policy is only as strong as the weakest client that connects.

2. Certificate stores and trust lists

Most OPC UA stacks keep their certificates in a folder-based PKI store with a standard layout: the application's own certificate and private key, a *trusted* folder, an *issuers* folder, and a *rejected* folder. Exact paths differ by vendor, but the folder names are recognizable.

Copy the certificate files (never the private keys) from each store and run them through the inventory tool from issue 01, `cert_inventory.py`. The `rejected` folder deserves a separate look. It records clients that tried to connect and were refused, which is often the fastest way to find OPC UA clients nobody documented.

Look for these in particular:

- **Long validity.** Many products generate a self-signed application certificate at install time with a validity of ten years or more. Those certificates are

the long-lived trust anchors that issue 01 put at the top of the list.

- **Self-signed certificates trusted individually.** Each one is a separate trust decision that must be revisited when algorithms change. A Global Discovery Server (GDS) or an internal CA that issues OPC UA certificates turns hundreds of individual decisions into one.
- **SHA-1 signatures and 1024-bit RSA keys.** These are already weak classically and are an easy first fix.

3. The traffic itself

OPC UA has a property that helps an inventory: the start of every secure channel is readable even when the channel is encrypted. The OpenSecureChannel request carries the security policy URI and the sender's certificate in its asymmetric security header, before any symmetric encryption begins.

So a passive capture of control-center traffic, taken from a SPAN port or tap under your normal change control, shows which policy each client and server actually negotiated and which certificate each one presented. Wireshark's OPC UA dissector decodes these headers. This gives you what configuration files cannot: the policies really in use, not just the ones on offer.

Channels are long-lived, and a capture only sees channels that open or renew during the capture window. Run the capture long enough to cover channel renewals, which are set by each application's token lifetime.

4. The software stack

Each OPC UA product embeds a protocol stack and a cryptographic library, often an OpenSSL build, the .NET cryptography APIs, or mbedTLS on embedded devices. Post-quantum support for OPC UA will arrive through those libraries first. Ask each vendor which stack and which

cryptographic library their product uses, and which version. That answer tells you more about the product's upgrade path than any roadmap slide.

What to record

Add these fields to the inventory from issue 01, one row per OPC UA application:

FIELD	WHY IT MATTERS
Application URI and role (server, client, or both)	Identifies the application independently of host name
Endpoints, with security modes and policies offered	What the application allows
Policy observed in traffic	What it actually uses
Certificate key type, size, signature algorithm, expiry	From `cert_inventory.py`
Certificate issuer: self-signed, GDS, or internal CA	Determines how certificates get replaced
Stack and cryptographic library, with versions	Determines how algorithms get replaced
Can the policy be changed by configuration?	Separates a settings change from a firmware or product change

What to do now

1. **Remove deprecated policies.** Basic128Rsa15 and Basic256 use SHA-1 and should not appear on any production endpoint.
2. **Close `None` endpoints** on production servers, or document exactly why each one remains.
3. **Standardize on one current policy** across the control center where products allow it. Fewer policies in use means fewer places to change

later.

4. **Move certificate issuance to a GDS or an internal CA**, so that a future algorithm change is a re-issuance rather than hundreds of manual trust decisions.
5. **Shorten the longest-lived application certificates** when you next touch each system.
6. **Ask every OPC UA vendor in writing** which stack and cryptographic library they ship, whether they support the OPC UA 1.05 ECC policies, and what their plan is for post-quantum security policies.

None of this waits for a post-quantum OPC UA standard. The work that makes a future migration fast is the same work that makes the current deployment cleaner: fewer policies, fewer exceptions, and a certificate authority instead of a collection of self-signed trust decisions.

Views are my own and do not represent any employer or client. This post uses only public sources and contains no non-public operator information.

Sources

- OPC Foundation, *OPC 10000-2: OPC Unified Architecture Part 2, Security Model*, and *OPC 10000-7: Part 7, Profiles* (security policy definitions), version 1.05.
- PHP-OPCUA documentation, Security policies (policy list and deprecation status).
- The White House, Securing the Nation Against Advanced Cryptographic Attacks (EO 14412), June 22, 2026.
- Ikwuogu, F. O., "What EO 14412 means for a pipeline operator's certificate inventory," *Quantum-Safe Pipeline Notes*, issue 01, 2026. https://doi.org/10.5281/zenodo.23130001

Text licensed CC BY 4.0. Cite as: Friday Ogochukwu Ikwuogu, “Finding the cryptography inside an OPC UA deployment,” Quantum-Safe Pipeline Notes, issue 3.